Programming With C Sharp

April 2024

Q 1 (A) Attempt the follow **4**

1. MSIL stands for____. - **Microsoft Intermediate Language**
2. CTS stands for____ . - **Common Type System**
3. Which namespace is fundamental for basic C# operation. - **System namespace**
4. Array of array is called_____. - **Jagged Array**

(B) Answer in brief(Any One) **2**

1. Explain garbage collection.

In C#, garbage collection is an automatic process managed by the Common Language Runtime (CLR) to handle memory allocation and reclamation. When you create objects in C#, they are allocated on the managed heap. The garbage collector (GC) periodically identifies and frees memory occupied by objects that are no longer accessible, thus preventing memory leaks and optimizing available memory.

2. What is assembly module.

In C#, an **assembly** is the fundamental unit of deployment, version control, and security for managed applications. It encapsulates a collection of types and resources designed to function cohesively, typically manifesting as a single .exe or .dll file.

Within an assembly, there exists a concept known as a **module**. A module is a unit of compilation that contains type metadata and compiled code. While most assemblies consist of a single module, it's possible to have multi-module assemblies where multiple modules are combined into one assembly.

(C) Answer in details(Any One) **3**

1. What is JIT? Explain types of JIT.

In the context of the .NET framework, a **Just-In-Time (JIT) compiler** plays a crucial role in executing applications. When you write code in languages like C# or VB.NET, it's initially compiled into an intermediate language known as Microsoft Intermediate Language (MSIL) or Common Intermediate Language (CIL). The JIT compiler's responsibility is to convert this intermediate code into native machine code specific to the computer's architecture at runtime

Types of JIT Compilers in .NET:

The .NET framework employs different types of JIT compilers to cater to various performance and resource requirements:

1. **Normal JIT Compiler:**
   - **Functionality:** Compiles methods on-demand when they are called during execution. Once a method is compiled, the native code is stored in memory for subsequent calls, reducing compilation overhead.
2. **Econo JIT Compiler:**
   - **Functionality:** Similar to the Normal JIT but with a focus on conserving memory. It compiles methods as they are called but does not store the compiled code in memory after execution, freeing up resources.
3. **Pre-JIT Compiler:**
   - **Functionality:** Compiles the entire MSIL code into native code in a single compilation cycle, typically at the time of deployment. This process is facilitated by tools like the Native Image Generator (NGen).

2. Explain Data Types In C#.Net.

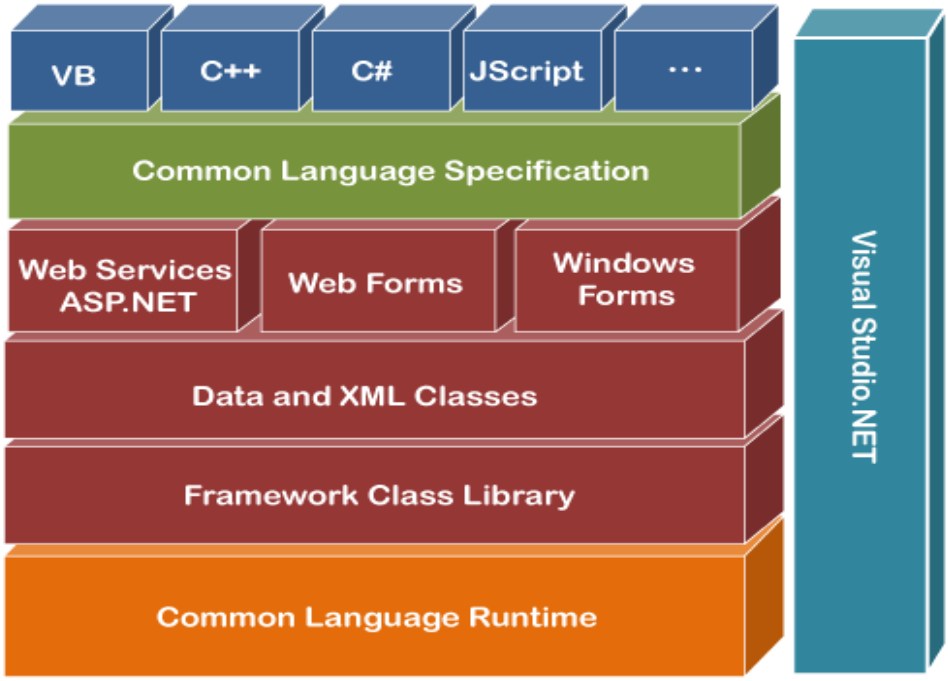
In C#, **data types** define the kind of data a variable can hold, specifying its size and the operations that can be performed on it. Understanding data types is fundamental for effective memory management and application performance.

Data Type	Size	Description
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647.
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.
float	4 bytes	Stores fractional numbers, sufficient for storing 6 to 7 decimal digits.
double	8 bytes	Stores fractional numbers, sufficient for storing 15 decimal digits.
bool	1 byte	Stores true or false values.
char	2 bytes	Stores a single character/letter, surrounded by single quotes.
string	Varies	Stores a sequence of characters.

(D) Write a note on following question:(Any One) **5**

1. Explain architecture of .Net Framework.

The .NET Framework is a comprehensive and consistent programming model developed by Microsoft for building applications with visually stunning user experiences, seamless and secure communication, and the ability to model a range of business processes. Its architecture is designed to provide a managed execution environment, simplified development, and integration with a variety of programming languages.



Key Components of the .NET Framework Architecture:

1. **Common Language Runtime (CLR):**
   - The CLR is the execution engine for .NET applications. It provides services such as memory management, type safety, exception handling, garbage collection, security, and thread management.
   - Code compiled for the .NET Framework is converted into an intermediate language called Common Intermediate Language (CIL). During execution, the

CLR's Just-In-Time (JIT) compiler translates this CIL code into native machine code specific to the underlying architecture.

2. **.NET Framework Class Library (FCL):**
    - The FCL is a comprehensive collection of reusable classes, interfaces, and value types that provide access to system functionality. It is organized into a hierarchy of namespaces and includes classes for tasks such as file reading and writing, graphic rendering, database interaction, and XML document manipulation.
    - The FCL extends the Base Class Library (BCL) by adding libraries for specific functionalities like Windows Forms, ASP.NET, and Windows Presentation Foundation (WPF).
3. **Common Language Infrastructure (CLI):**
    - The CLI provides a language-neutral platform for application development and execution. By implementing core aspects of the .NET Framework within the scope of the CLI, these functions are available across multiple programming languages supported by the framework.
4. **Assemblies:**
    - Assemblies are the building blocks of .NET applications. They are compiled code libraries used for deployment, versioning, and security. An assembly can consist of one or more files, and it contains a manifest that provides metadata about the assembly's contents.

2. Explain for loop and for each loop with example.

In C#, loops are fundamental constructs that allow you to execute a block of code repeatedly. Two commonly used loops are the for loop and the foreach loop. Understanding their syntax and use cases is essential for effective programming.

1. for Loop:

The for loop is typically used when the number of iterations is known beforehand. It consists of three parts: initialization, condition, and iteration.

Syntax:

```
for (initialization; condition; iteration)
{
   // Code block to be executed
}
```

Initialization: Executed once at the beginning and is used to declare and set the loop control variable.

Condition: Evaluated before each iteration; if it evaluates to true, the loop continues; if false, the loop terminates.

Iteration: Executed after each iteration and is typically used to update the loop control variable.

Example:

```
for (int i = 0; i < 5; i++)
{
   Console.Write(i);
}
```

Explanation:

- **Initialization:** int i = 0; sets the loop control variable i to 0.
- **Condition:** i < 5; the loop runs as long as i is less than 5
- **Iteration:** i++ increments i by 1 after each loop iteration.

Output:0 1 2 3 4

2. foreach Loop:

The foreach loop is used to iterate over elements in a collection (e.g., arrays, lists) without needing an explicit loop control variable. It's particularly useful when the number of elements is unknown or when direct access to each element is required.

Syntax:

```
foreach (type variableName in collection)
{
   // Code block to be executed
}
```

type: The data type of the elements in the collection.

variableName: A temporary variable that holds the current element during each iteration.

collection: The collection to iterate over.

Example:

```
string[] cars = { "Volvo", "BMW", "Ford", "Mazda" };
foreach (string car in cars)
{
   Console.Write(car);
}
```

Explanation:

- An array cars is defined with four car brand names.
- The foreach loop iterates over each element in the cars array, assigning the current element to the variable car.
- Inside the loop, Console.Write(car); prints the current car brand.

Output: Volvo BMW Ford Mazda

Q 2 (A) Attempt the follow **4**

1._____class will prevent inheritance. - **Sealed Class**

2._____collection ensures that all elements are unique. - **HashTable**

3. C# does not support _____Inheritance. - **Multiple inheritance**

4. The BitArray class is presented in_____Namespace**.** - **System.Collections**

(B)Answer in brief:(Any One) **2**

<u>1.What is method overloading?</u>

In C#, **method overloading** allows multiple methods in the same class to share the same name but differ in their parameter lists (number, type, or order of parameters). This enables the creation of methods that perform similar functions with different inputs, enhancing code readability and flexibility.

<u>2. Explain get and Set properties.</u>

In C#, **properties** provide a flexible mechanism to encapsulate class fields, offering controlled access through **get** and **set** accessors. This approach aligns with the principle of **encapsulation** in object-oriented programming, allowing data to be accessed and modified safely.

Structure of Properties:

A property comprises two primary accessors:

- **get accessor:** Retrieves the property's value.
- **set accessor:** Assigns a new value to the property.

These accessors enable the creation of read-only, write-only, or read-write properties, depending on the inclusion of get and set.

```
public class Person
{
   private string name; // Private backing field
   public string Name   // Property
   {
      get { return name; }
      set { name = value; }
   }
}
```

(C) Answer in detail.(Any One) **3**

1. What is Interface? Explain with example.

In C#, an **interface** is a contract that defines a set of methods, properties, events, or indexers without implementing them. Classes or structs that implement this interface are required to provide concrete implementations for all its members. Interfaces facilitate a form of multiple inheritance, allowing a class or struct to inherit behaviors from multiple sources, promoting flexibility and polymorphism in your code.

Defining an Interface:

An interface is declared using the interface keyword. By convention, interface names often begin with an uppercase 'I'.

```
public interface IShape
{
   void Draw();
   double GetArea();
}
```

2. Explain ArrayList with Example.

In C#, an **ArrayList** is a dynamic collection provided by the System.Collections namespace that allows you to store elements of varying data types. Unlike arrays, which have a fixed size, an ArrayList can dynamically grow or shrink as elements are added or removed.

Creating an ArrayList:

To utilize an ArrayList, you must first include the System.Collections namespace:

```
using System.Collections;
ArrayList myList = new ArrayList();
```

Adding Elements:

You can add elements to an ArrayList using the Add() method:

```
myList.Add(42);          // Adding an integer
myList.Add("Hello");     // Adding a string
myList.Add(3.14);        // Adding a double
myList.Add(true);        // Adding a boolean
```

Inserting Elements:

To insert an element at a specific index, use the `Insert()` method:

```
myList.Insert(1, "World"); // Inserts "World" at index 1
```

Removing Elements:

Elements can be removed by value using `Remove()` or by index using `RemoveAt()`:

```
myList.Remove(3.14);     // Removes the element with value 3.14
myList.RemoveAt(0);      // Removes the element at index 0
```

(D) Write a note on following question(Any One) **5**

1. Define Delegate. Explain Multicast delegation with example.

In C#, a **delegate** is a type that defines a method signature, allowing methods to be passed as parameters. Delegates enable the creation of references to methods with specific parameter lists and return types, facilitating callback methods and event handling.

Defining a Delegate:
To declare a delegate, use the `delegate` keyword followed by the method signature:
public delegate void Notify(string message);
This declaration defines a delegate named `Notify` that can reference any method with a `void` return type and a single `string` parameter
Instantiating and Using a Delegate:
Once a delegate is defined, you can instantiate it and assign a method that matches its signature:

```
public class Process
{
   public static void DisplayMessage(string msg)
   {
      Console.WriteLine(msg);
   }
}

class Program
{
   static void Main()
   {
      Notify notifier = new Notify(Process.DisplayMessage);
      notifier("Hello, World!");
   }
}
```

MULTICAST DELEGATES: Delegate's ability to multicast means that a delegate object can maintain a list of methods to call, rather than a single method if you want to add a method to the invocation list of a delegate object , you simply make use of the overloaded += operator, and if you want to remove a method from the invocation list you make use of the overloaded operator -=

```
using System;
public delegate void MulticastDelegate(int x, int y);
public class MyClass
{
    public static void Add(int x, int y)
    {
        Console.WriteLine("You are in Add() Method");
        Console.WriteLine("{0} + {1} = {2}\n", x, y, x + y);
    }
    public static void Multiply(int x, int y)
    {
        Console.WriteLine("You are in Multiply() Method");
        Console.WriteLine("{0} X {1} = {2}", x, y, x * y);
    }
}
class Program
{
    static void Main(string[] args)
    {
        MulticastDelegate del = new MulticastDelegate(MyClass.Add);
        del += new MulticastDelegate(MyClass.Multiply);
        Console.WriteLine("calling Add() and Multiply() Methods.");
        del(5, 5);
        del -= new MulticastDelegate(MyClass.Add);
        Console.WriteLine("\n****Add() Method removed.****\n");
        del(5, 5);
    }
}
```

```
calling Add() and Multiply() Methods
You are in add() method
5 + 5 =10
You are in multiply() method
5 X 5 =25
****Add() Method removed.****
You are in multiply() method
5 X 5 =25
```

2. What is Event? Explain with example.

In C#, an **event** is a mechanism that enables a class (known as the **publisher**) to notify other classes or objects (known as **subscribers**) when something of interest occurs. Events are built upon **delegates**, which define the method signature for the event handlers that respond to the event.

Understanding Events:

- **Delegates:** A delegate is a type that represents references to methods with a specific parameter list and return type. Delegates are used to define the signature of event handler methods.
- **Events:** An event is a declaration within a class that is associated with a delegate. It acts as a conduit through which the publisher can notify subscribers when a particular action occurs.
- **Implementing Events in C#:**

1. **Define a Delegate:** Specify the signature for the event handler methods.
   public delegate void ProcessCompletedEventHandler(string message);
   2. Declare the Event: Use the `event` keyword along with the delegate to declare the event within the publisher class.
   public class Process
   {
   public event ProcessCompletedEventHandler ProcessCompleted;
   }
   3 Raise the Event: Create a method within the publisher class to invoke the event, typically named `OnEventName`.

```
public class Process
{
   public event ProcessCompletedEventHandler ProcessCompleted;
   public void StartProcess()
   {
      Console.WriteLine("Process Started.");
      // Simulate process work...
      OnProcessCompleted("Process Completed Successfully.");
   }
   protected virtual void OnProcessCompleted(string message)
   {
      ProcessCompleted?.Invoke(message);
   }
}
```

Q 3 (A) Attempt the follow **4**

1. _____name space is required for windows application. -**System.Windows.Forms namespace.**
2. _____ method is used to load the content of a file in RichTextBox. - **LoadFile**
3. _____control is combination of TextBox and ListBox. -**ComboBox**
4. MDI stands for _____. - **Multiple Document Interface**

(B)Answer in Brief(Any One) **2**

1. Write a short note on mouse Event.

In C#, mouse events are integral to handling user interactions within graphical user interfaces (GUIs). These events enable applications to respond to various mouse actions, such as movements, clicks, and wheel scrolling. Both Windows Forms and Windows Presentation Foundation (WPF) frameworks provide a set of mouse events to facilitate this interaction.

Common Mouse Events:

1. **MouseDown:** Triggered when a mouse button is pressed over a control.
2. **MouseUp:** Occurs when a mouse button is released over a control. **Click:** Raised when the mouse is pressed and released on the same control.
3. **DoubleClick:** Fired when the user quickly clicks the mouse button twice on a control. **MouseMove:** Occurs when the mouse pointer is moved over a control.
4. **MouseEnter:** Triggered when the mouse pointer enters the boundaries of a control. **MouseLeave:** Occurs when the mouse pointer leaves the boundaries of a control.
5. **MouseHover:** Raised when the mouse pointer hovers over a control for a specified duration.

2. Explain ProgressBar.

In C#, a **ProgressBar** is a user interface control that visually represents the progress of a lengthy operation. It provides users with feedback on the status of ongoing tasks, enhancing the user experience by indicating that the application is actively processing.

Key Properties:

- **Minimum:** Defines the lower bound of the progress range. Typically set to 0. **Maximum:** Sets the upper bound of the progress range, indicating when the task is complete.
- **Value:** Represents the current progress within the defined range.
- **Step:** Specifies the increment added to the Value property when the PerformStep() method is called.
- **Style:** Determines the visual behavior of the ProgressBar.

(C)Answer in Details:(Any One) **3**

1. What is MessageBox()? Explain With Example.

In C#, the MessageBox class provides a straightforward way to display a modal dialog box to the user, typically used for conveying information, warnings, or prompts that require user interaction.

Key Features of MessageBox:

- **Message Display:** Shows a specified text message to the user.
- **Title Bar Caption:** Allows customization of the dialog's title bar text.
- **Buttons:** Supports various button configurations (e.g., OK, OK/Cancel, Yes/No) to capture user responses.
- **Icons:** Can display standard icons (e.g., Information, Warning, Error, Question) to indicate the nature of the message.

Basic Usage Example:

To display a simple message box with an informational message:

```
MessageBox.Show("Operation completed successfully.", "Information",
MessageBoxButtons.OK, MessageBoxIcon.Information);
```

In this example:

- "Operation completed successfully." is the message displayed
- "Information" is the title bar captionMessageBoxButtons.OK specifies that the message box will have an OK button. MessageBoxIcon.Information displays an information icon alongside the message.

2. <u>Explain Treeview with properties and example</u>.

In C#, the **TreeView** control is utilized to display hierarchical data in a tree-like structure, allowing users to expand and collapse nodes to navigate through the data. This control is commonly employed to represent structures such as file directories, organizational charts, or any nested data relationships.

Key Properties of TreeView:

- **Nodes:** This property holds the top-level nodes in the TreeView. Each node is an instance of the TreeNode class, which can have its own child nodes, enabling the creation of a hierarchical structure.
- **SelectedNode:** Represents the currently selected node in the TreeView. You can get or set this property to determine or change the selection.
- **ImageList:** Assigns an ImageList to the TreeView, allowing nodes to display images alongside their labels.
- **CheckBoxes:** When set to true, each node will display a checkbox next to it, enabling multiple selections or indicating selection states
- **FullRowSelect:** Determines whether the selection highlight spans the entire width of the TreeView or just the node label.
- **ShowLines:** Indicates whether lines are drawn between nodes to represent the hierarchy visually.

```
private void Form1_Load(object sender, EventArgs e)
        {
           treeView1.Nodes.Add("BCA");
           treeView1.Nodes[0].Nodes.Add("SEM -4");
           treeView1.Nodes[0].Nodes[0].Nodes.Add("C sharp");
           treeView1.Nodes[0].Nodes[0].Nodes.Add("JAVA");
           treeView1.Nodes[0].Nodes[0].Nodes.Add("NETWORK");
           treeView1.Nodes[0].Nodes[0].Nodes.Add("UNIX LINUX");

        }
```

(D) Write a note on following question:(Any One) **5**

<u>1. Explain picture Box control with example</u>.

In C#, the **PictureBox** control is a Windows Forms component designed to display images in various formats, such as BMP, JPEG, GIF, PNG, and others. It provides a straightforward way to incorporate images into your application's user interface.

Key Properties of PictureBox:

- **Image:** Sets or retrieves the image displayed in the PictureBox.
  - **ImageLocation:** Specifies the path or URL of the image to display.
  - **SizeMode:** Determines how the image is displayed within the PictureBox. Options include
  - Normal: The image is placed in the upper-left corner of the PictureBox.

- StretchImage: The image is stretched or shrunk to fit the PictureBox
- AutoSize: The PictureBox adjusts its size to match the image's size.
- CenterImage: The image is centered within the PictureBox.
- Zoom: The image is proportionally resized to fit the PictureBox without distortion.

- **BorderStyle:** Sets the border style of the PictureBox, such as None, FixedSingle, or Fixed3D.

Example of Using PictureBox:

Here's how to create a PictureBox control, set its properties, and display an image:

```
using System;
using System.Drawing;
using System.Windows.Forms;
public class PictureBoxExample : Form
{
   private PictureBox pictureBox;

   public PictureBoxExample()
   {
      // Initialize the PictureBox
      pictureBox = new PictureBox
      {
         Width = 400,
         Height = 400,
         BorderStyle = BorderStyle.Fixed3D,
         SizeMode = PictureBoxSizeMode.StretchImage,
         Image = Image.FromFile("C:\\Images\\SampleImage.jpg") // Set your image path here
      };

      // Add the PictureBox to the form
      Controls.Add(pictureBox);
   }

   [STAThread]
   static void Main()
   {
      Application.EnableVisualStyles();
      Application.Run(new PictureBoxExample());
   }
}
```

2. Explain OPenFileDialog and FontDialog.

In C#, the OpenFileDialog and FontDialog are pre-configured dialog components provided by the Windows Forms library, facilitating user interactions for file selection and font customization, respectively.

OpenFileDialog

The OpenFileDialog component displays a standard dialog box that allows users to browse and select one or more files from the file system. It's commonly used in applications that require file input from users.

Key Properties:

- **InitialDirectory**: Sets the initial directory displayed when the dialog box is opened.
  Filter: Determines the types of files displayed (e.g., "Text files *(.txt)*/.txt|All files *(.)*/.").
- **FilterIndex**: Specifies the index of the filter currently selected in the dialog box.
  Multiselect: Indicates whether multiple files can be selected.
- **RestoreDirectory**: If set to true, the dialog box restores the current directory to its original value before closing.

```
string s;
DialogResult result = openFileDialog1.ShowDialog();
    if (result == DialogResult.OK) // Test result.
     {
         s = openFileDialog1.FileName;
         MessageBox.Show(result.ToString());
         string ss = File.ReadAllText(s);
         textBox1.Text = ss;
    }
```

FontDialog

The FontDialog component presents a dialog box that enables users to select a font and its attributes, such as style, size, and effects. It's particularly useful in text-editing applications where font customization is desired.

Key Properties:

- **Font**: Gets or sets the selected font.
- **ShowColor**: Determines whether the dialog box displays the option to select a color.
- **Color**: Gets or sets the selected color if ShowColor is enabled.
- **AllowScriptChange**: Indicates whether the dialog box allows users to change character scripts (writing systems).

```
if ( fontDialog.ShowDialog() != DialogResult.Cancel )
{
textBox1.Font = fontDialog.Font;
 }
```

Q 4(A)Attempt the follow **4**

1. ADO stands for____. - **ActiveX Data Objects**
2. To connect with sqlserver _____ namespace is used. - **System.Data.SqlClient**
3. _____ is a bridge in between DataSet and database. - **DataAdapter**
4._____ method is used to clear all tables and constarints from the DataSet. - **Reset**

(B)Answer in Brief:(Any One) **2**

1.<u>Explain DataGridView.</u>

A DataGridView is a flexible and powerful control used in Windows Forms applications (commonly in .NET using C# or VB.NET) to display, edit, and manage tabular data. It's like a spreadsheet-style table where each row represents a data record and each column represents a data field.

Where It's Used:

- Windows Forms (WinForms) applications
- Common in database-driven applications
- Useful for displaying lists, reports, or editable data tables

Key Features:

- **Automatic data binding**: You can bind it to data sources like a DataTable, BindingList, or a database.
- **Customizable columns**: Text, checkbox, combo box, image, and button columns.
- **Built-in editing**: Users can edit cells directly in the grid.
- **Sorting and filtering**: Basic support for sorting; custom filtering can be added.
- **Events**: Many events like CellValueChanged, RowValidated, and CellClick for interactive behavior.

2. **<u>What is ExplainDataTable?</u>**

A DataTable is used to store, retrieve, and manipulate tabular data (like a spreadsheet or SQL table) within your application **without needing a database connection**.

Structure:

A DataTable contains:

- **Columns** – define the schema (names and data types)
- **Rows** – hold the actual data
- Optional: **Constraints**, **Primary keys**, and **Relationships** with other tables

Common Uses:

Holding results from a **SQL query** (SqlDataAdapter.Fill() can load data into a DataTable)

- Passing structured data around in an app (e.g., between layers)
- Binding to UI controls like **DataGridView** or **DataGrid**

(C) Answer in detail: (Any One) **3**

1<u>. Explain DataAdapter with example.</u>

In **C#**, a DataAdapter acts as a **bridge** between a **database** and an **in-memory DataTable or DataSet**. It allows you to:

- Fetch data from a database (using SELECT)
- Fill that data into a DataTable or DataSet
- Optionally update the database using INSERT, UPDATE, DELETE commands

DataAdapter.Fill() = Pull data from DB into memory

DataAdapter.Update() = Push changes from memory back to DB

```
public DataSet SelectDB2SrvRows(DataSet dataset,string connection,string query)
{
   DB2Connection conn = new DB2Connection(connection);
   DB2DataAdapter adapter = new DB2DataAdapter();
   adapter.SelectCommand = new DB2Command(query, conn);
   adapter.Fill(dataset);
   return dataset;
}
```

<u>2.Explain ExecuteNonQuery() and ExecuteScalar().</u>

ExecuteNonQuery()

Purpose:

Used for **SQL statements that don't return data**, such as:

- INSERT
- UPDATE
- DELETE
- CREATE, ALTER, DROP (DDL)

Return Value:

Returns an int — the **number of rows affected** by the SQL command.

```
using (SqlConnection conn = new SqlConnection("your_connection_string"))
{
   conn.Open();
   string query = "UPDATE Employees SET Age = 35 WHERE ID = 1";
   SqlCommand cmd = new SqlCommand(query, conn);

   int rowsAffected = cmd.ExecuteNonQuery();
   Console.WriteLine($"Rows affected: {rowsAffected}");
}
```

ExecuteScalar()
Purpose:
Used for SQL queries that return **a single value** — typically:

- SELECT COUNT(*)
- SELECT MAX(Salary)
- SELECT Name FROM Employees WHERE ID = 1

Return Value:
Returns an object — the **first column of the first row** in the result set.
(You may need to cast or convert it.)

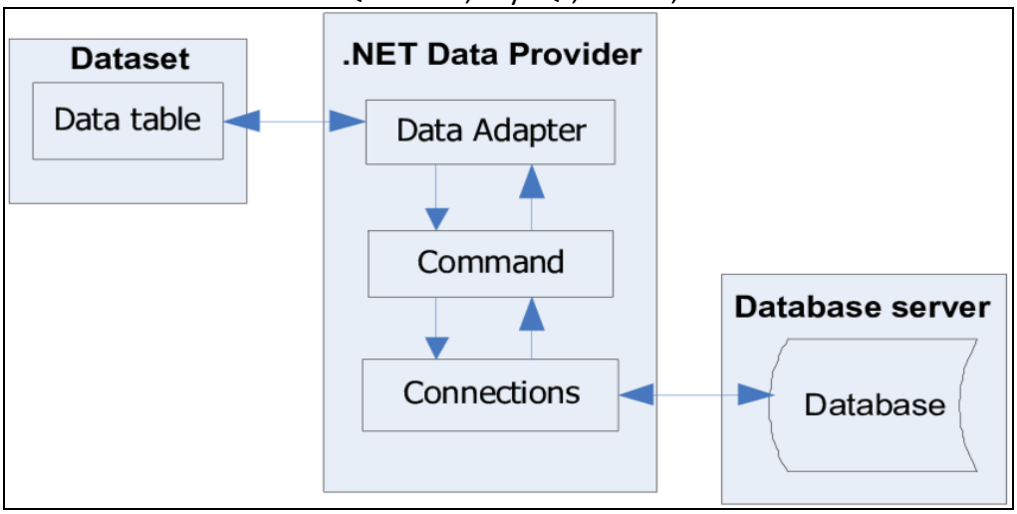
```
using (SqlConnection conn = new SqlConnection("your_connection_string"))
{
   conn.Open();
   string query = "SELECT COUNT(*) FROM Employees";
   SqlCommand cmd = new SqlCommand(query, conn);

   int employeeCount = (int)cmd.ExecuteScalar();
   Console.WriteLine($"Employee count: {employeeCount}");
}
```

(D) Write a note on following question:(Any One) **5**

1. Explain ADO.NET Architecture in detail.

ADO.NET (ActiveX Data Objects .NET) is a set of classes in the .NET Framework that allows applications to **connect to data sources**, **retrieve**, **manipulate**, and **update** data — typically from relational databases like SQL Server, MySQL, Oracle, etc.



ADO.NET Architecture is a part of the .NET Framework designed to handle data access from various sources such as SQL Server, Oracle, and XML. It provides a consistent and efficient way to connect, retrieve, manipulate, and update data. The architecture is divided into two main models: **Connected** and **Disconnected**.

Connected Architecture
This model maintains a continuous connection with the data source during operations. It is best suited for real-time, read-only access to large amounts of data.

Key components:

- **SqlConnection**: Establishes and manages a connection to the database.
- **SqlCommand**: Executes SQL queries or stored procedures.
- **SqlDataReader**: Provides fast, forward-only, read-only access to data.
- **SqlTransaction**: Manages transactions for executing multiple operations atomically.

Disconnected Architecture
This model retrieves data from the database, stores it in memory, and closes the connection, allowing manipulation of data offline. It's suitable for multi-tier or distributed applications.
Key components:

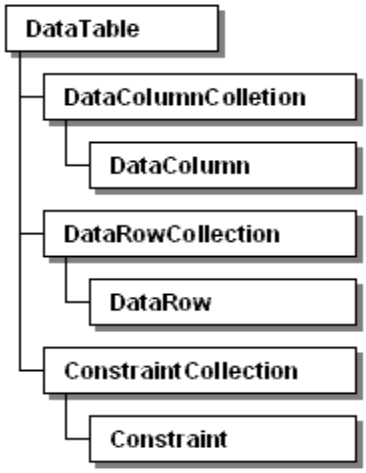
- **DataAdapter**: Acts as a bridge between the database and in-memory data (DataSet/DataTable). It handles data retrieval (Fill) and updates (Update).
- **DataSet**: An in-memory cache that can store multiple DataTable objects and maintain relationships between them.
- **DataTable**: Represents a single table of in-memory data.

2. Explain DataRow and DataColumn properties and methods.
What is DataColumn?
A **DataColumn** represents a **column** in a DataTable. It defines the schema — including the **name**, **data type**, and **constraints** (like AllowDBNull, Unique, etc.).

```
DataTable table = new DataTable();
DataColumn col = new DataColumn("Age", typeof(int));
col.AllowDBNull = false;
table.Columns.Add(col);
```

Common Properties:

- ColumnName: Name of the column.
- DataType: Type of data (e.g., typeof(int)).
- AllowDBNull: Whether the column allows null values.
- Unique: Ensures all values are unique.
- DefaultValue: Sets a default value for the column.



What is DataRow?
A **DataRow** represents a **single row** of data in a DataTable. You can use it to add, retrieve, and modify data in individual cells.

```
DataRow row = table.NewRow();
row["Age"] = 30;
table.Rows.Add(row);
// Read value
int age = (int)row["Age"];
```

Common Properties:

- ItemArray: Gets or sets all the values in the row as an array.
- RowState: Shows the state of the row (e.g., Added, Modified).

Common Methods:

- BeginEdit() / EndEdit(): Start and stop batch editing.
- IsNull(columnName): Checks if a value is null.
- SetField<T>() / Field<T>(): Generic methods to get/set column values.

Q 5 (A) Attempt the follow **4**

1. User Control is based on____ - **System.Web.UI.UserControl**
2. Which control of form is used to view the Crystal Report? - **CrystalReportViewer**
3._____ event will occur whenever the user control loads. - **Load**
4. Section 3 for crystal report includes___ - **Detail Section**

(B) Answer in Brief :(Any One) **2**

1. <u>List various types of reports.</u>

Here's a list of various types of reports:

1. Standard Report
2. Cross-Tab Report
3. Chart Report
4. Subreport
5. Parameterized Report
6. Summary Report
7. Detail Report
8. Drill-Down Report
9. Invoice or Form Report
10. Mailing Label Report

2. <u>List out types of setup projects.</u>

Types of Setup Projects in C#:

1. **Setup Project (MSI Installer)**
   - Creates a .msi installer using Visual Studio Installer.
2. **Web Setup Project**
   - Deploys ASP.NET web applications to IIS servers.
3. **Merge Module Project (.msm)**
   - Used to package shared components to be included in multiple setup projects.
4. **ClickOnce Deployment**
   - Simplifies deployment and automatic updates via a web or network location.
5. **WiX (Windows Installer XML) Project**
   - Uses XML to define setup logic and create MSI installers; supports advanced customization.
6. **InstallShield Project**
   - Commercial tool integrated with Visual Studio for creating professional installers.

(C) Answer in details(Any One) **3**

1. <u>Explain User Control with property</u>.

A **User Control** in C# is a custom, reusable UI component that combines multiple existing controls like labels, buttons, and textboxes into one unit. It's built by inheriting from the UserControl class, commonly used in **Windows Forms** or **ASP.NET** applications.

Creating a User Control (Windows Forms Example)

Suppose you create a UserControl called MyUserControl that contains a Label

```
public partial class MyUserControl : UserControl
{
```

```
    public MyUserControl()
    {
        InitializeComponent();
    }
    // Custom Property
    public string Title
    {
        get { return label1.Text; }
        set { label1.Text = value; }
    }
}
```

Explanation:

- label1.Text is a control inside the user control.
- The public property Title allows external forms or classes to **get/set** the label's text.

Using the User Control in a Form

2. Explain special fields available in crystal report.

Special Fields **in** Crystal Reports are predefined fields provided by the report engine to display system-generated or report-related information. These fields are automatically populated and do not require database data**.**

Common Special Fields:

1. **Print Date**
    - Displays the date when the report is printed or previewed.
2. **Print Time**
    - Shows the exact time the report was printed or viewed.
3. **Modification Date**
    - Shows the last date the report file (.rpt) was modified.
4. **Report Title**
    - Displays the title of the report, defined in report settings.
5. **Page N of M**
    - Shows pagination like: *Page 1 of 5*.
6. **Page Number**
    - Displays the current page number.

Usage:

- Insert these from **Field Explorer → Special Fields**.
- Drag and drop them into headers, footers, or detail sections as needed.
- No code or database connection is needed for these fields.

(D) Write a note on following question :(Any One) 5

1. Explain All sections of crystal reports.

Sections in Crystal Reports

Crystal Reports divides a report into **logical sections** to help organize and control how data is displayed. These sections are processed in a specific order when the report is generated.

In **Crystal Reports**, a standard report is divided into five primary sections, each serving a distinct purpose in organizing and displaying data:

1. **Report Header**:
    - **Purpose**: Appears once at the beginning of the report.
    - **Usage**: Ideal for the main title, company logo, or introductory information.
2. **Page Header**:
    - **Purpose**: Appears at the top of every page.

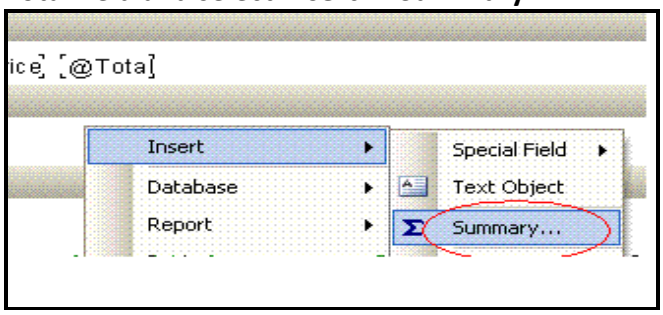
- **Usage**: Typically contains column headings or page-specific titles to label the data presented on that page.

3. **Details**:
   - **Purpose**: Displays the main body of the report.
   - **Usage**: Shows the actual data records retrieved from the database, with each record typically occupying one line.
4. **Report Footer**:
   - **Purpose**: Appears once at the end of the report.
   - **Usage**: Used for grand totals, summaries, or concluding remarks.
5. **Page Footer**:
   - **Purpose**: Appears at the bottom of every page.
   - **Usage**: Commonly includes page numbers, print dates, or other contextual information relevant to the entire page.

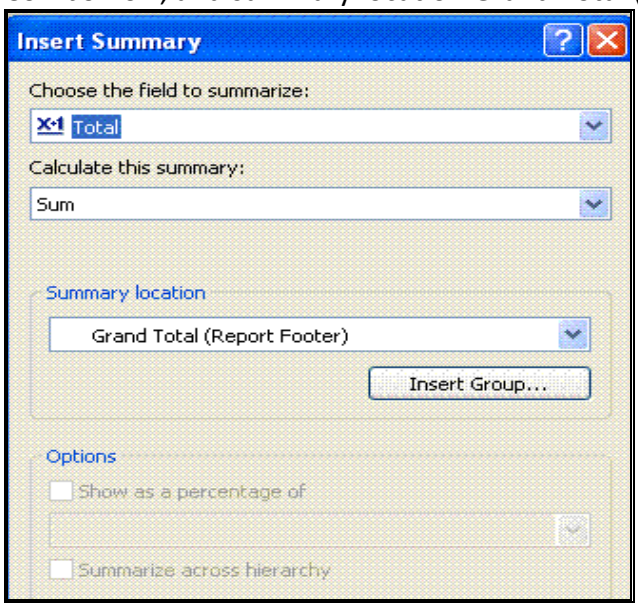
2. <u>Explain summary field and formula field in crystal report.</u>

Summary Field

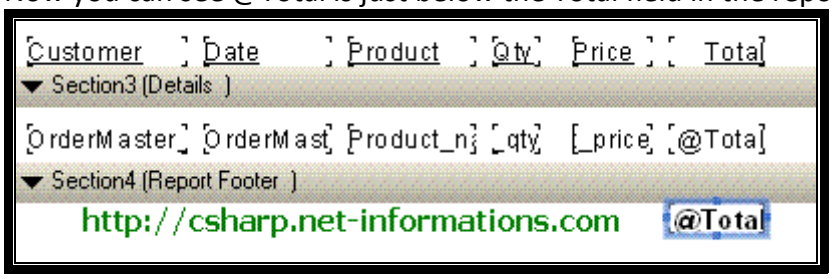
In the Crystal Reports designer view window, right click on the Report Footer , just below the Total field and select **Insert -> Summary** .



Then you will get a screen , select the Total from the combo box and select Sum from next Combo Box , and summary location Grand Total (Report Footer) . Click Ok button



Now you can see @Total is just below the Total field in the report Footer.



Now the designing part is over and the next step is to call the Crystal Reports in C# and view it in Crystal Reports Viewer control .

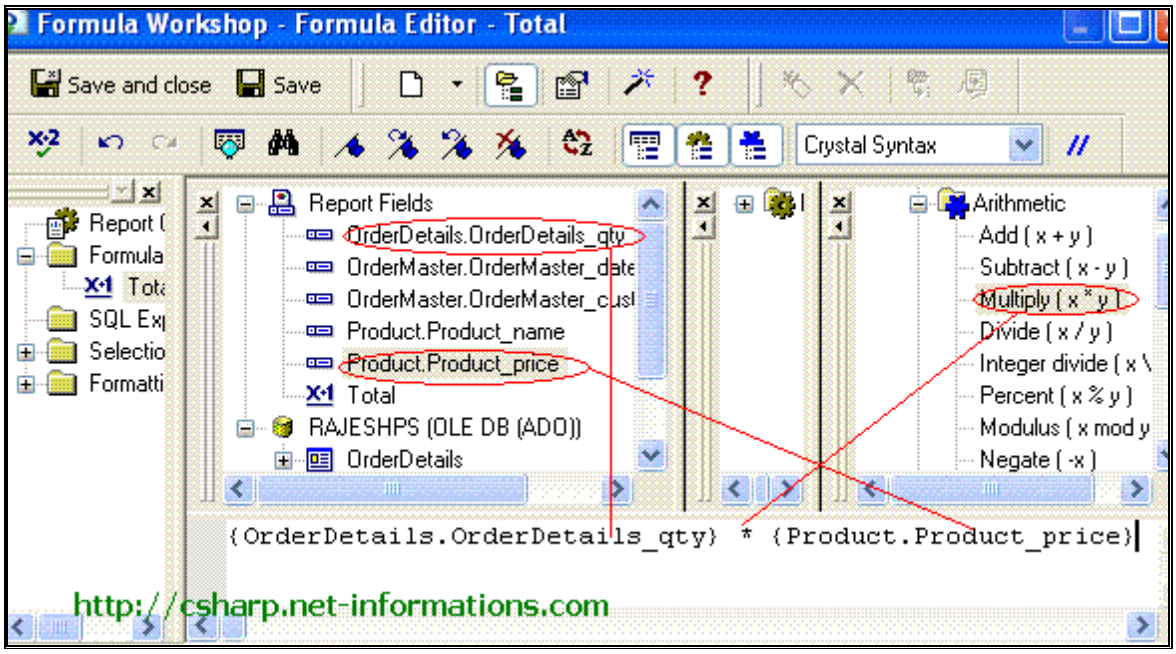
Select the default form (Form1.cs) you created in C# and drag a button and a CrystalReportViewer control to your form .

FORMULA FIELD IN CRYSTAL REPORT:

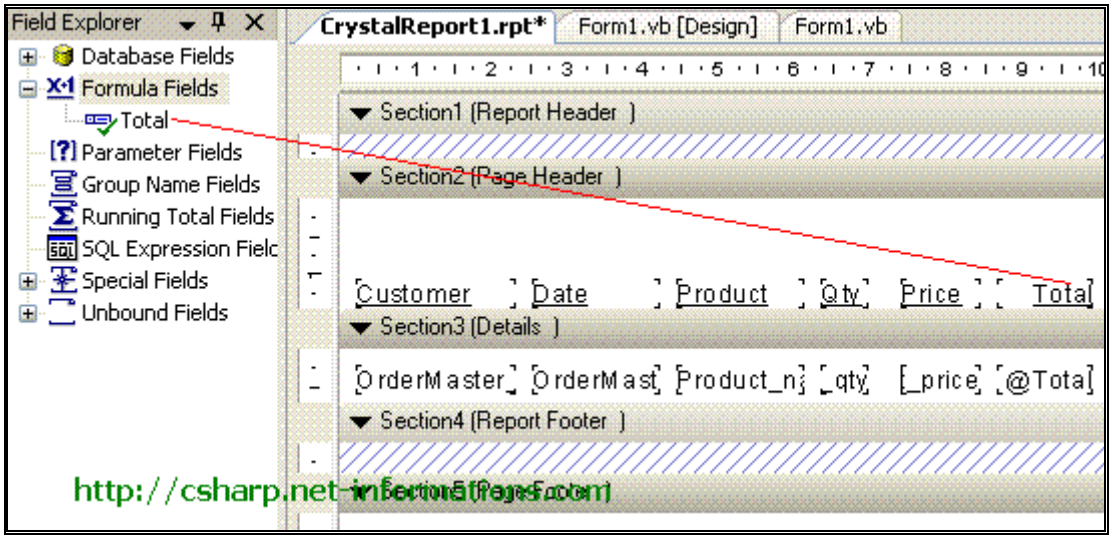
If you have a Crystal Reports with Qty and Price fields and you need an additional field in your Crystal Reports for Total, that is **TOTAL = QTY X PRICE** . In these types of situations you can use the Formula Field in Crystal Reports.

In this C# Crystal Reports tutorial we are showing that all orders with qty and price and the total of each row , that means each in each row we are showing the total of qty and price.

Before starting this tutorial Create a new Crystal Reports with fields CustomerName , Order Date , Product Name and Product Price . If you do not know how to create this report , just look the previous tutorial C# Crystal Reports from multiple tables . In that report selecting only four fields , here we need one more field Product->Price and one formula field Total.



Now you can see Total Under the Formula Field . Drag the field in to the Crystal Reports where you want to display Total.



Now the designing part is over and the next step is to call the Crystal Reports in C# and view it in Crystal Reports Viewer control .

Select the default form (Form1.cs) you created in C# and drag a button and a CrystalReportViewer control to your form .